

DESIGN PATTERNS

Prompt Smells

From Code Smells to Prompt Design , Recognizing, Diagnosing, and Refactoring Bad Prompts

9

Teaching-focused
prompt smells

2

Research
foundations (2024 /
2026)

1

Refactoring mindset,
borrowed from
code

From Code Smells to Prompt Smells

- **What is a code smell?**

A surface sign in source code that something may be wrong beneath , not a bug, but a warning.

- **Familiar example: Long Method**

A method doing too much is hard to read, test, and reuse. Developers recognize it on sight.

- **How developers respond**

They refactor: split responsibilities, clarify names, isolate behavior , without changing what the code does.

- **The same idea, applied to prompts**

A prompt can also carry warning signs. Ronanki et al. (2024) call these prompt smells.

CODE

Bad code → Long Method → refactored code

One giant method handling validation, calculation, and output is split into small, named, single-purpose methods.

PROMPT

Bad prompt → Prompt Bloat → refactored prompt

An over-decorated persona and buried request are reduced to a direct, well-scoped instruction.

A prompt smell is a surface-level indicator in a prompt's wording or structure that correlates with a higher risk of an undesirable AI response.

A Prompt Smell Is a Warning Sign , Not a Bug

1

A prompt can smell and still work

A prompt with a smell may still produce a good answer. The model can compensate.

2

A clean prompt can still fail

A prompt with no obvious smell can still produce a poor, generic, or off-target answer.

3

So why does this matter?

Smells raise the probability of a bad outcome , shifting the odds is why prompt quality needs evaluation, not just intuition.

CLASSROOM PROMPT

“Tell me about databases.”

Ask the class:

Is this wrong? No.

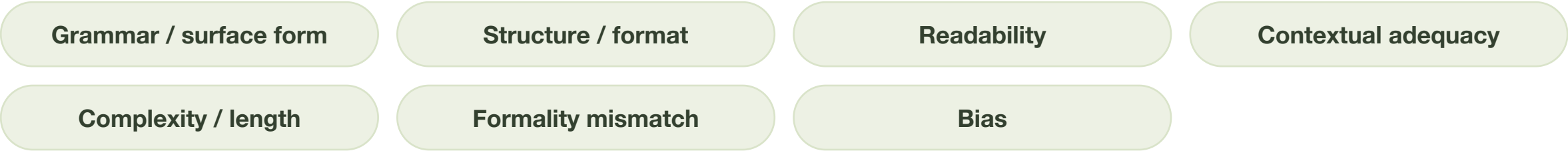
Is it well specified? Probably not.

A Research-Based View of Prompt Smells

2024 Ronanki, Cabrero-Daniel & Berger introduce “prompt smells” as an omen for undesirable generative-AI outputs , the founding concept.

2026 Della Porta et al. investigate how to measure prompt quality, proposing a preliminary taxonomy of prompt-smell dimensions.

PRELIMINARY RESEARCH TAXONOMY (2026)



Our nine teaching examples are not the research taxonomy itself , they are a teaching classification derived from these research dimensions, chosen for how clearly they map onto familiar code-smell intuitions.

PART I

Context & Clarity Smells

1 Vague / Missing Context

2 Ambiguous References

3 Format Ambiguity

Vague / Missing Context

CONTEXT

A teacher asks an AI assistant to prepare a lecture.

WHY IT'S A PROBLEM

- Who is the audience?
- What level of study?
- What aspects should be covered?
- How much detail is expected?

SMELLY PROMPT

“Explain software architecture.”



REFACTORED PROMPT

“Explain software architecture to second-year Software Engineering students. Cover layered, client-server, and microservices architectures, with one practical example for each.”

Ambiguous References

CONTEXT

A software project meeting is being summarized for the next steps.

WHY IT'S A PROBLEM

- Who is “they”?
- What is “it” referring to?
- Which problem , of possibly several?

Rule of thumb: if a pronoun could refer to more than one noun in the prompt, replace it with the noun.

SMELLY PROMPT

“The team changed it because they found a problem. Explain what they should do next.”



REFACTORED PROMPT

Name the project, the team, the specific artifact that changed, and the specific problem found , before asking what should happen next.

Format Ambiguity

CONTEXT

A manager wants a project-risk report from the AI assistant.

WHY IT'S A PROBLEM

- No output shape is specified
- “Some recommendations” has no count or structure
- Priority ordering is left to chance

Takeaway: output format is part of prompt design, not an afterthought.

SMELLY PROMPT

“Analyze these project risks and give me some recommendations.”



REFACTORED PROMPT

“Create a table with four columns: Risk, Probability, Impact, and Recommended Mitigation. Rank the risks from highest to lowest priority.”

PART II

Complexity Smells

- 1 Overloaded Prompt
- 2 Prompt Bloat / Convolved Prompt
- 3 Unnecessary Repetition
- 4 Conflicting Constraints

Overloaded Prompt

CONTEXT

A student is developing a library management system.

WHY IT'S A PROBLEM

- Analogous to a method doing too many things
- Each phase needs different context to do well
- Errors in early phases silently corrupt later ones

SMELLY PROMPT

“Design the requirements, UML diagrams, database, architecture, Java implementation, test cases, documentation and presentation for a library system.”



REFACTORED PROMPT

Split into a sequence: Requirements → Use cases → Architecture → Database → Implementation → Testing , one prompt, and one reviewed output, per stage.

Prompt Bloat / Convoluted Prompt

CONTEXT

A user wants a Factory Method example for a design-patterns course.

WHY IT'S A PROBLEM

- The real request is buried at the end
- Decorative framing adds no task-relevant information
- Longer prompts are harder to verify against the actual need

SMELLY PROMPT

“Imagine that you are a world-class software architect who has spent 30 years working with enterprise systems and has advised governments and Fortune 500 companies...” , eventually , “Give me a Factory Method example in Java.”



REFACTORED PROMPT

“Give me a simple Java example demonstrating the Factory Method pattern. Explain the problem it solves and identify the participants.”

Unnecessary Repetition

CONTEXT

A student asks the AI to explain inheritance.

WHY IT'S A PROBLEM

- Same instruction repeated in different words
- No new information is added by repeating
- Directly analogous to duplicate code

Connect to code smells: this is duplicate code, restated as instructions instead of statements.

SMELLY PROMPT

“Explain inheritance in simple language. Keep it simple. Make it easy. Don’t make it complicated. Use simple words and a simple example.”



REFACTORED PROMPT

“Explain inheritance using simple language and one Java example for beginners.”

Conflicting Constraints

CONTEXT

Writing an executive report for stakeholders.

WHY IT'S A PROBLEM

- “Comprehensive” and “exactly 100 words” pull in opposite directions
- The model must silently choose which constraint to break
- Tension is invisible until the output disappoints

Refactoring resolves the tension by naming exactly what “comprehensive” must include within the limit.

SMELLY PROMPT

“Write a comprehensive report covering all aspects of the project in exactly 100 words.”



REFACTORED PROMPT

“Write a 100-word executive summary covering the project’s objective, three key findings, major risk, and recommendation.”

PART III

Semantic & Quality Smells

- 1 Irrelevant / Excessive Persona
- 2 Bias / Loaded Framing
- 3 Formality / Audience Mismatch

Irrelevant / Excessive Persona

CONTEXT

A simple programming question about Java interfaces.

WHY IT'S A PROBLEM

- The persona does not change what a correct answer looks like
- Extra tokens spent on framing, not on the task
- Ask: does this persona actually help perform the task?

SMELLY PROMPT

“Act as a world-renowned Nobel Prize-winning software architect, cybersecurity expert, university professor, CEO and technology visionary... What is a Java interface?”



REFACTORED PROMPT

Keep a persona only when it changes the desired response , otherwise, ask the question directly: “What is a Java interface?”

Bias / Loaded Framing

CONTEXT

Choosing a technology approach for a new system.

WHY IT'S A PROBLEM

- “Obviously better” presumes the conclusion
- The analysis is biased before it begins
- Undermines objective, evidence-based decision-making

Connects directly to unbiased requirements engineering and defensible decision-making.

SMELLY PROMPT

“Why is microservices obviously better than monolithic architecture?”



REFACTORED PROMPT

“Compare monolithic and microservices architectures for a university ERP system. Evaluate scalability, development complexity, deployment, cost and maintainability.”

Formality / Audience Mismatch

CONTEXT

Explaining cybersecurity concepts to senior government officers.

WHY IT'S A PROBLEM

- Tone is mismatched to a formal, high-stakes audience
- Casual register can undercut credibility
- Directly named in the 2026 research taxonomy

SMELLY PROMPT

“Yo, let’s break down zero trust and see why it’s kinda awesome.”



REFACTORED PROMPT

“Explain Zero Trust Architecture in formal but accessible language for senior government officials, using a government information-system example.”

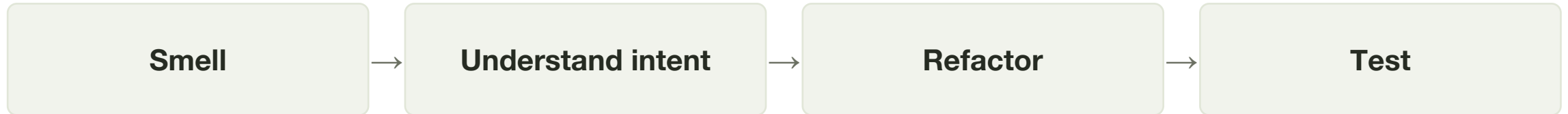
PART IV

Prompt Refactoring

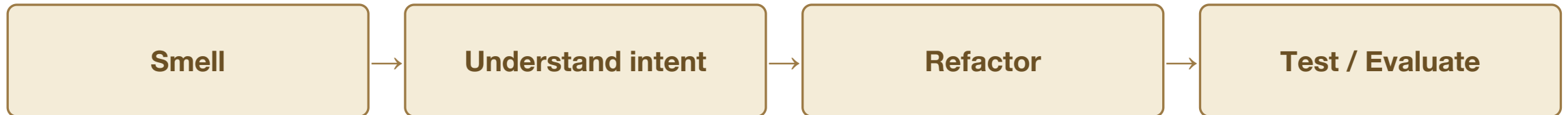
- 1 The Prompt Refactoring Mindset
- 2 A Practical Refactoring Pattern
- 3 Full Worked Example
- 4 Mapping to Software Design Principles

The Prompt Refactoring Mindset

CODE

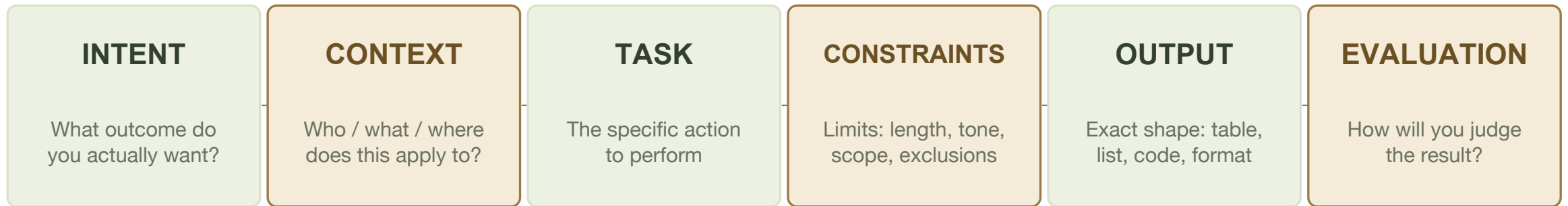


PROMPT



Just like refactoring never changes what a program is supposed to do, prompt refactoring never changes what you're asking for , only how clearly, completely, and unambiguously you ask for it.

A Practical Refactoring Pattern



Then transform one bad prompt step-by-step, adding exactly one missing stage at a time so students can see which change fixed what.

EXAMPLE APPLICATION

“Give me a Java payment example using a design pattern.” → applying INTENT (clear pattern comparison) + CONTEXT (varying payment methods) + TASK (implement Strategy) + OUTPUT (class structure + runnable example) yields a fully specified prompt , see the worked example next.

Full Example: From Bad Prompt to Good Prompt

SCENARIO: A university course-registration system

v1 , Smelly

“Design a good modern secure course registration system for our university.”

v2 , Add context

+ Institution type, student population, existing systems it must integrate with.

v3 , Clarify task

+ Specific deliverable: e.g. “propose the system architecture and core entities.”

v4 , Add constraints

+ Security requirements, scale, technology constraints, timeline.

v5 , Define output

+ “Provide a component diagram, a data model, and a short risk list.”

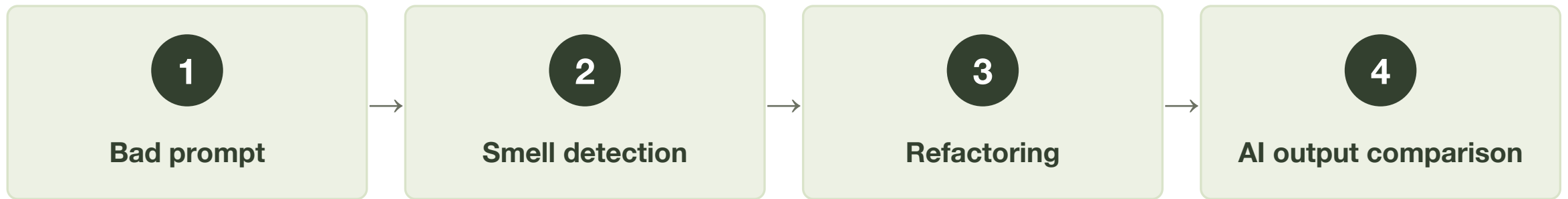
Students watch the prompt being refactored incrementally , exactly like code , and see which single addition changes the output most.

Prompt Smells ↔ Software Design Principles

SOFTWARE DESIGN	PROMPT DESIGN
Single Responsibility	One clear objective
High Cohesion	Related instructions kept together
Low Coupling	Avoid unnecessary dependencies between asks
Encapsulation	Provide only relevant context
Interface / Contract	Explicit, agreed output format
Refactoring	Prompt improvement

Case Study: Requirements Engineering

Give students a realistic requirements-engineering scenario and walk the full cycle end to end.



WHY THIS MATTERS FOR SOFTWARE ENGINEERING STUDENTS

Requirements engineering already trains students to interrogate ambiguity, missing stakeholders, and conflicting constraints. Running that same discipline on a prompt , instead of a requirements document , makes prompt quality feel like a natural extension of a skill they already have, not a new one.

Case Study: Programming / Design Pattern

“Write code for a payment system using a suitable design pattern.”

Students identify the ambiguity first: which pattern? which payment methods? what does “suitable” mean here?

REFACTORED

“Design a Java payment-processing example where the payment method can vary between CreditCard, bKash, and PayPal. Use the Strategy pattern. Provide the class structure and a small runnable example.”

Now students can see how design-pattern thinking , naming the varying behavior, choosing Strategy, specifying participants , improves prompts the same way it improves code.

Classroom Activity , Prompt Smell Detector

Give each group 5 prompts. For each one:

1 Identify the smell

2 Explain the context

3 Explain why it is problematic

4 Refactor it

5 Generate outputs using both

6 Compare the results

Goal: make smell-spotting and refactoring a repeatable habit , the same habit students already use during code review.

Evaluation: Did Refactoring Actually Help?

01

Correctness

02

Relevance

03

Completeness

04

Format adherence

05

Consistency

06

Robustness

07

Traceability

Students evaluate the smelly and refactored outputs against these criteria , turning “the refactored prompt feels better” into a defensible, structured comparison.

Research Limitations

- ! Prompt-smell research is still emerging
- ! The 2026 taxonomy is preliminary, not final
- ! Smells are context-dependent , the same phrasing can smell in one setting and not another
- ! A smell does not guarantee a bad output
- ! Removing a smell does not guarantee a good output

Treat the catalogue in this deck as a teaching tool for building intuition , not as a certified, exhaustive checklist.

KEY TAKEAWAYS

Designing Prompts Is a Design Discipline

- ✓ Prompt smells are warning signs, not verdicts.
- ✓ Context matters , the same prompt can be fine or flawed depending on it.
- ✓ Clear prompts reduce ambiguity for both humans and models.
- ✓ Complex prompts can often be decomposed, just like complex methods.
- ✓ Output requirements should be made explicit, not assumed.
- ✓ Prompt refactoring is directly analogous to code refactoring.
- ✓ Good prompts are designed, inspected, and refined , not merely written.

REFERENCES

Sources

2024

Ronanki, K., Cabrero-Daniel, B., & Berger, C. (2024). Prompt Smells: An Omen for Undesirable Generative AI Outputs.

2026

Della Porta et al. (2026). Toward Measuring Prompt Quality: A Preliminary Investigation on Prompt Smells.

Thank you

Questions and discussion welcome.